

SWE40006

Software Deployment and Evolution

Lecture 5
Software (SDLC) evolution
CI/CD and CD
Containers

Where are we now?

We covered,

- Activities within software deployment (Carzaniga Model)
- Deployment approaches (manual vs. automated), with additional focus on data migration
- A framework to plan deployments

Related Learning objectives

1. Describe the activities in software deployment and apply these in a problem context.
2. Apply knowledge of software environments to plan development and deployment.
3. **Describe and classify issues that drive software maintenance.**
4. Describe latest practices in software development deployment and evolution.
5. Draw on industry practice to evaluate the maintainability and design consistency of a software system.

How is software built?

- Software is built by making a number of decisions
 - Decisions are made with incomplete information
 - We evolve software into a “good enough” structure and then eventually shape/adapt it
 - Decisions are made with “human rationality”
- Evolution of software starts the day you have a first (syntax) error-free build
 - It does not have to do much -- just compile cleanly

Construction, Maintenance & Evolution

- **Traditional view** is that we 'identify a problem'
 - Design a solution
 - Construct it
- At some point in time, this is released to end-users
- After this point in time -- this software is *maintained*.
- Maintenance causes the software system to slowly evolve over time.
- Gaining features, and progressively becomes *issue-free*.
- So ... is this how developers actually work?

Detour - Defect, Issue or Bug?

- A piece of advice from experience:
 - Software has 'issues' (or) 'bugs'.
 - Avoid using the word 'defect'
- Why? The reasons are mainly 'legal' and 'how normal people relate to the word defect'
- Example: You have been told that the DVD player that you just purchased has over 100 known defects -- will you buy this product?
 - Legally this product can be considered 'defective' by a court of law and manufacturer held liable
- Most commercial 'bug tracking software' are called 'issue tracking system' (same reason)

Software Construction

- Independent of the methodology used in a project the following statements hold:
- Software is constructed incrementally
- Components/Features/Functions are built -- verified and then integrated
- Initial solutions are 'approximately correct' -- they are modified to improve accuracy
- A software system gains features as it evolves to meet user expectations (which may or may not be captured in a requirements document)
- Every decision made during construction is a 'trade-off'

Areas that change during maintenance

- Functionality → Source code or config. file changes
- Documentation → Comments or external documents
- Non-Functional Properties → Performance, targeting 64-bit CPU

Refactoring

- Refactoring is a maintenance activity
 - Involves making changes to existing code
 - The changes do not alter the observable Functional behaviour
 - Changes internal structure of software
 - Change may be to a small part of the program
- Refactoring is undertaken to
 - Improve code readability (reduce technical debt)
 - Consolidate architecture (help for later change)
 - May Improve performance (side-effect)

Re-engineering

- Re-engineering often involves major parts of the system (or all of it)
- The system is dis-assembled and then re-assembled to,
 - Improve performance
 - Introduce “new” features
 - Provide support for or to take advantage of newer technologies
- Change is ‘systemic’
- Higher risk and expensive

Re-structuring

- Restructuring implies:
 - The internal structure of the code is modified
- Refactoring involves 'restructuring'
- Re-engineering also often involves 're-structuring'
- When developers refactor, they are expected to write tests and check that the 'observable functional behaviour has not changed'

Refactor vs. Re-engineering

- Is this Just academic nit-picking?
 - Sadly, not in this case ...
- If a developer says “I am refactoring my code” there is an implied ‘purpose’
- The statement - “I am going to restructure my code” does not say “why”.
- The word “re-engineering” sets off alarm bells because it means massive and potentially systemic changes
- So ... choose your words carefully – even if the audience may not know, it is better to be precise

Types of Change causing Evolution

- Four things can happen to a class between two versions causing evolution and growth:
 - deleted
 - added
 - changed
 - untouched
- The same applies to a *method* or *package* (or component/namespace etc.)

Probability of Change

- The following is the order of probability.
- A class is most likely to be:
 - Unchanged
 - Modified
 - Newly created
 - Deleted (less than 1% Chance)
- The probability of modification decreases with age.

SWE40006

Software Deployment and Evolution

Lecture 5b

Contemporary deployment systems and Development pipelines

Software Deployment - Story so far...

- We covered the activities within software deployment
- We also covered the methods that can be applied to deploy (manual vs automated) -- and selection drivers
- In this lecture, we will look at the evolution of development pipelines which integrate areas such as development, testing, operations, security and deployment.

Related Learning objectives

1. Describe the activities in software deployment and apply these in a problem context.
2. Apply knowledge of software environments to plan development and deployment.
3. Describe and classify issues that drive software maintenance.
4. **Describe latest practices in software development deployment and evolution.**
5. Draw on industry practice to evaluate the maintainability and design consistency of a software system.

Lecture overview

This lecture will start to cover:

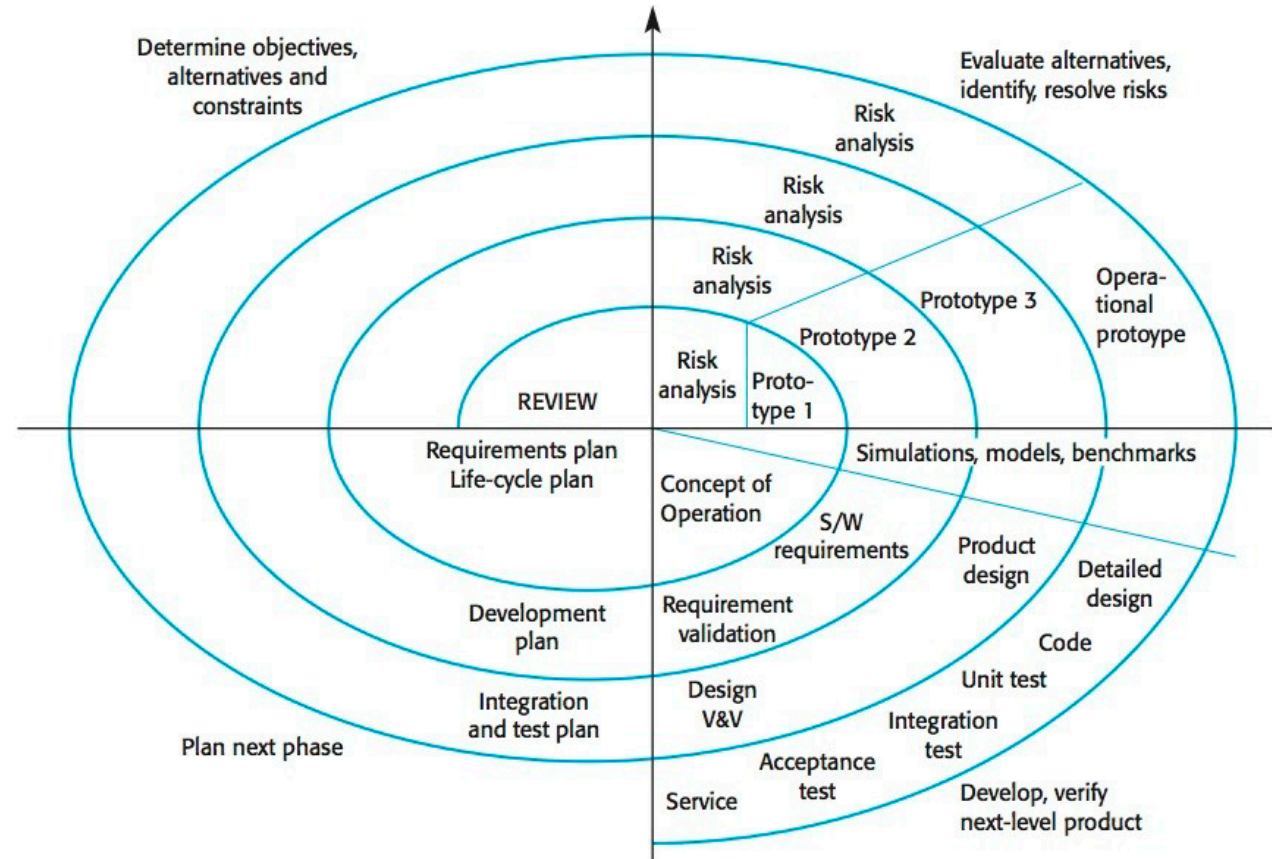
- History of SDLCs and development pipelines
- Extreme programming and the Agile manifesto (revision?)
- DevOps, DevSecOps
- CI/CD, Continuous Deployment and Continuous Delivery
- Containers
- IoT deployment, Games deployment and the future...

A brief history...

- 1970: Waterfall Model
- 1988: Spiral Model (Boehm, B. (1988), "A Spiral Model of Software Development and Enhancement," *IEEE Computer* 21, 5, 61-72.)
- 1991: Continuous Integration <https://www.martinfowler.com/articles/continuousIntegration.html>
- 1998: Carzinger
- 1999: Extreme Programming <https://www.computerworld.com/article/2585634/app-development/extreme-programming.html>
- 2001: Agile Dev <http://agilemanifesto.org/>
- 2009: DevOps Bass, Len; Weber, Ingo; Zhu, Liming. *DevOps: A Software Architect's Perspective*.
- 2009: Continuous Deployment & Continuous Delivery <https://puppet.com/blog/continuous-delivery-vs-continuous-deployment-what-s-diff>
- 2012: DevSecOps <http://www.devsecops.org/>
- 2015: CI/CD (Continuous Integration/Continuous Delivery) <https://dzone.com/articles/what-is-cicd>

Boehm's Spiral 1988

- 4 abstract waterfall stages:
 - Objectives,
 - risks,
 - product,
 - plan.
- repeat over and over again
- Gradually add scope
- Prototypes evolve into *prod* version.



<http://iansommerville.com/software-engineering-book/web/spiral-model/>

Continuous Integration 1991

- Developers in the team frequently integrate (build) their work.
- All the integrations are verified by an automated build server.
 - Build and test the code and detect integration errors.
- Encourages developers to merge their branches even after the completion of small tasks.
 - Shares their code and their unit tests with the rest of the team.
- Automated build system is triggered when the code is committed.
 - Grabs the latest version of the code from the shared repository and builds, tests and validates the master branch.

Continuous Integration benefits...

- Rapid feedback which ensures errors are fixed quickly, increasing productivity.
- Always know that the code will either pass integration testing or can be fixed quickly if it fails a test.
- Developers know what features need to be fixed.
- Many trivial (and easily fixed) bugs instead of “deferred integration” where bugs can accumulate to the point where refactoring or re-engineering become necessary*.
 - *although re-engineering may be required because of architecture design faults or changes

Continuous Integration

- Supports geographically distributed teams because integration is happening constantly ensuring that everyone is working on the same version of the problem.
- Code rollbacks cause loss of fewer code updates (compared to deferred integration) because of the frequent and automated testing.
- Developers are also testers (write their own test cases) and take responsibility for their work.
- Can be combined with Continuous Deployment, where current build is deployed to customers or servers every time a successful integration test occurs.

CI requirements

Build/test server - Automated testing scripts

<http://www.pepgotesting.com/continuous-integration/>

CI Server - notifies developers of test results

Continuous Integration Server

Source Control Server

Version (source) control repository - with no forks, just temporary branches

2 Fetch Changes

Check In Changes 1

Multiple developer workstations - Automated commits

6 Notify Success or Failure

Manager

Developer 1

Developer 2



CI process details*

1. The developer creates a new (temporary) branch out of the master branch and switches (checkout) into the new (temporary) branch. This new branch should be used for one change only with the intent of using the new branch for no longer than a few hours.
2. The developer implements the change in the new branch.
3. The developer runs a build process in the Development environment(s) with execution of the Unit Tests and the integration and automated Acceptance Tests (from Behaviour-Driven Development – BDD) that can run in the development environment(s). There can be one or multiple Development environments, depending on the requirements and complexity of the project.
4. The developer commits the change (to save the new branch locally).
5. The developer publishes the new branch (to upload the new branch in the master repository).
6. The developer sends a pull request to a senior (lead) developer.
7. The senior (lead) developer does a (code) review of the change.
8. If the senior (lead) developer is satisfied with the change, then the senior (lead) developer merges the new branch into the main branch and triggers a build and deployment process into the pre-production or production environment (for Continuous Delivery).

*<http://www.pepgotesting.com/continuous-integration/>

CI Tools

- Jenkins
- Team City
- Travis CI
- Visual Studio Team Foundation Server
- Bamboo
- U-build, U-deploy, U-release
- Go CD
- GitLab
- CircleCI
- Codeship
- Buddy
- Werker
- ...

Extreme programming (XP) 1999

- Features:
 - Frequent releases, short development cycles, pair programming, code review, unit testing.
 - Unlike old waterfall models these things all happen continuously; not after predetermined deadlines.
 - Code is the product (not working software). Code is used to explain ideas (because it is a superior form of communication).
 - Unit testing, acceptance testing is automated and tests all possible code paths and inputs.
 - Frequent refactoring, common coding standards, collective ownership of the code.
- Evolved from spiral SDLCs into Agile methods.

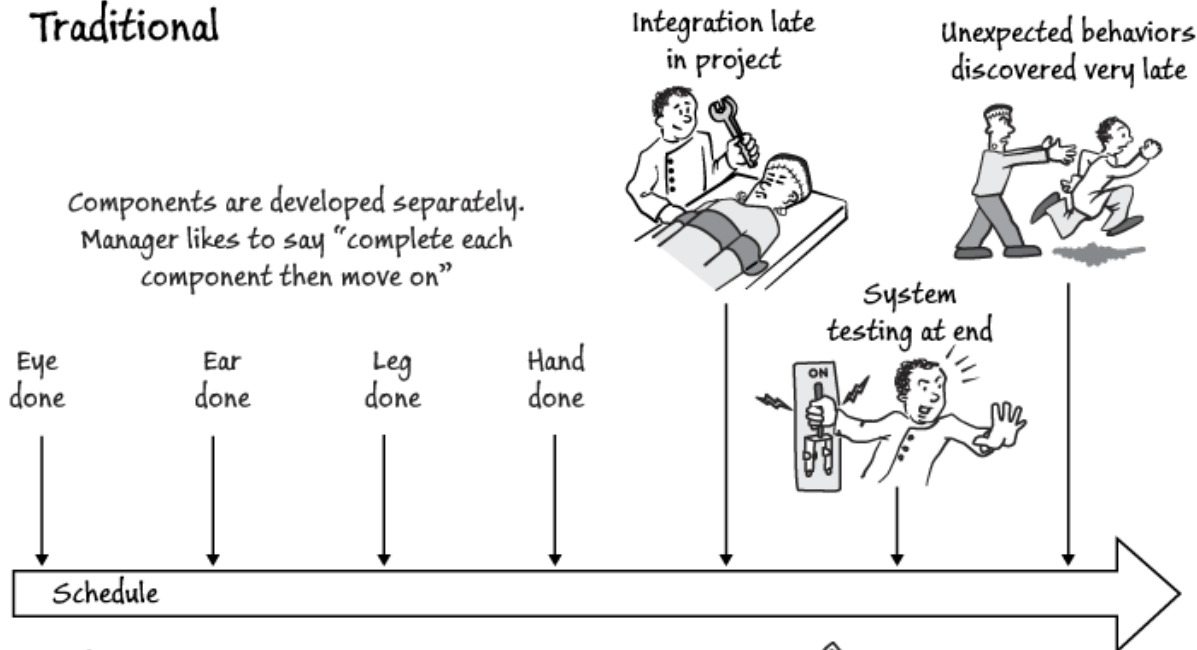
The Agile manifesto 2001

1. Our highest priority is to satisfy the customer through early and **continuous delivery** of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness **change** for the customer's competitive advantage.
3. Deliver working software **frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must **work together** daily throughout the project.
5. Build projects around motivated individuals. Give them the **environment and support** they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. **Working software** is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a **constant pace** indefinitely.
9. **Continuous** attention to technical excellence and good design enhances agility.
10. Simplicity--the art of maximizing the amount of work not done--is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

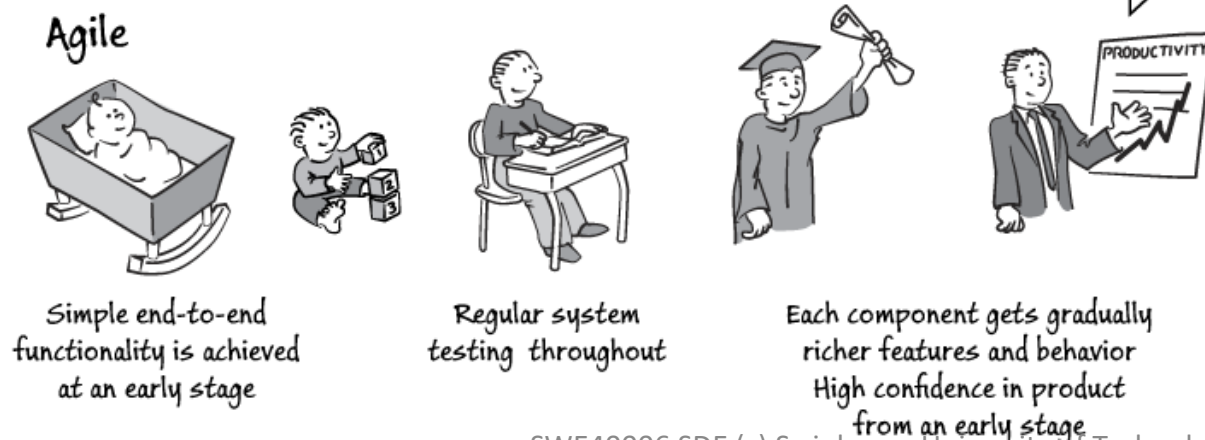
<http://agilemanifesto.org/principles.html>

Traditional vs. Agile Development

Traditional



Agile



Waterfall versus Agile development, <http://msdn.microsoft.com/en-us/library/jj159336.aspx>

DevOps 2009

- Development team = software developers, testers, maintenance, documentation, deployment.
- Operations team = management, monitoring and operation of production software.
- In DevOps we combine these teams.
 - Stops the “It’s not my problem” problem
 - Common tools increase productivity
 - Dev people consider Ops issues while coding
 - Ops people can program and fix things on the spot
 - Division between Dev and Ops blurred and all DevOps people take ownership of the product and have a stake in it’s success.

DevOps CALMS framework

- Culture – common goals, common ownership and responsibility
 - Automation – reduces workload and risk by automating testing
 - Lean – Use Agile methods/principles (not part of the “CAMS” framework)
 - Measurement – use Ops monitoring tools to detect flaws and drive improvements
 - Sharing - common goals, common ownership and responsibility*
- *yes, this is repeated (because it’s important).

DevOps CALMS framework



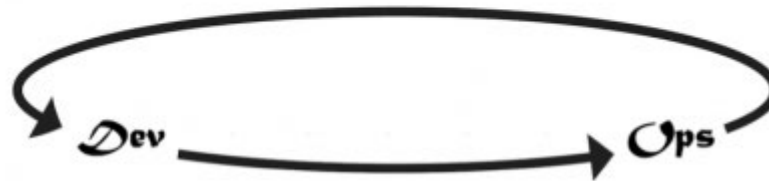
DevOps "The Three Ways"

<https://itrevolution.com/the-three-ways-principles-underpinning-devops>

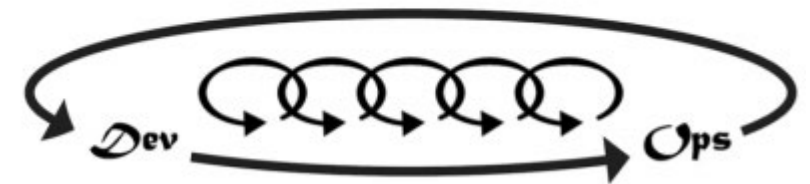
**The First Way:
Systems Thinking**



**The Second Way:
Amplify Feedback Loops**

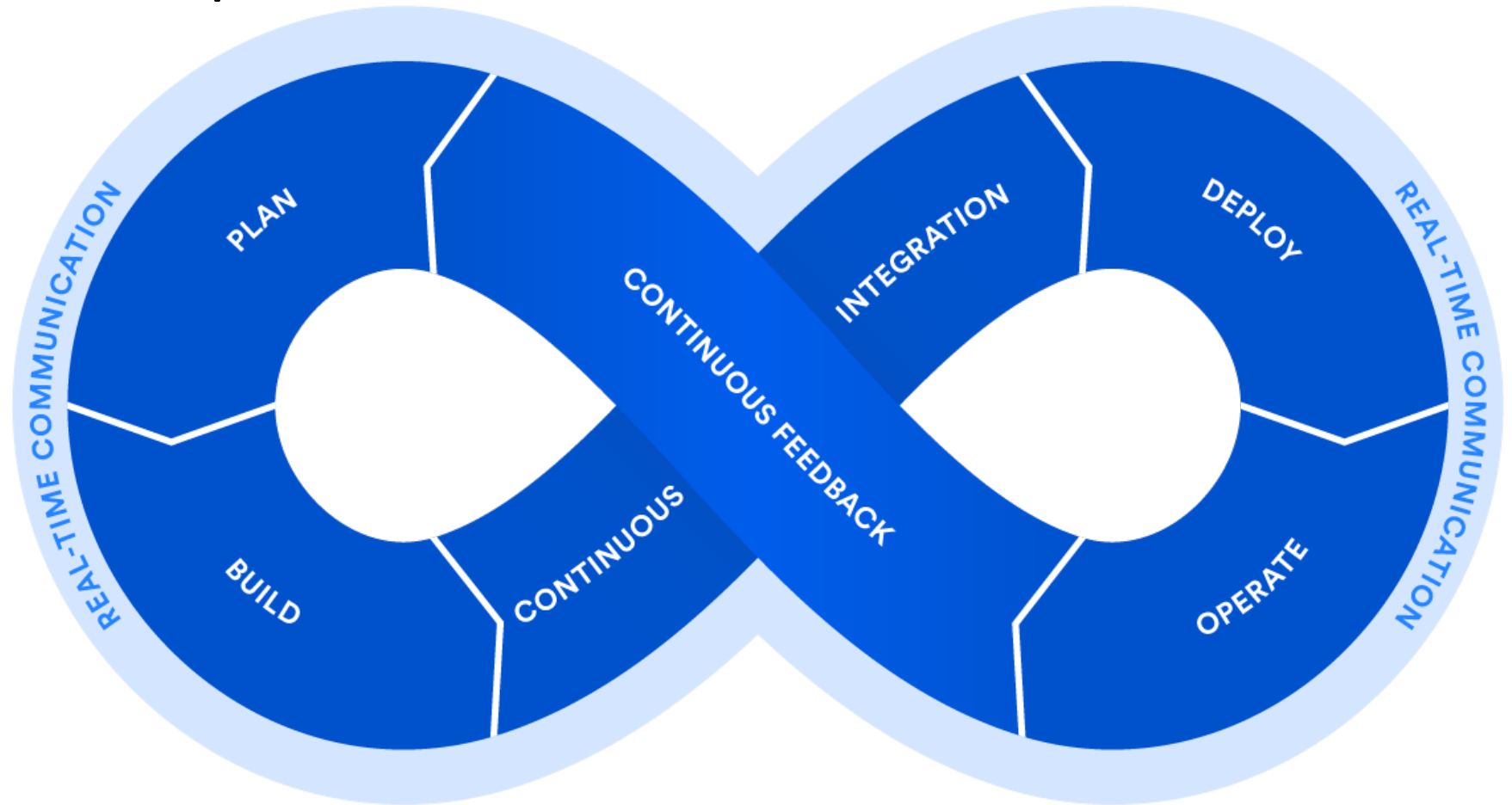


**The Third Way:
Culture Of Continual Experimentation And
Learning**



Stages of DevOps

- Dev:
 - Plan
 - Code
 - Build
 - Test
- Ops
 - Release
 - Deploy
 - Operate
 - Monitor



<https://www.atlassian.com/devops>

Who uses DevOps?

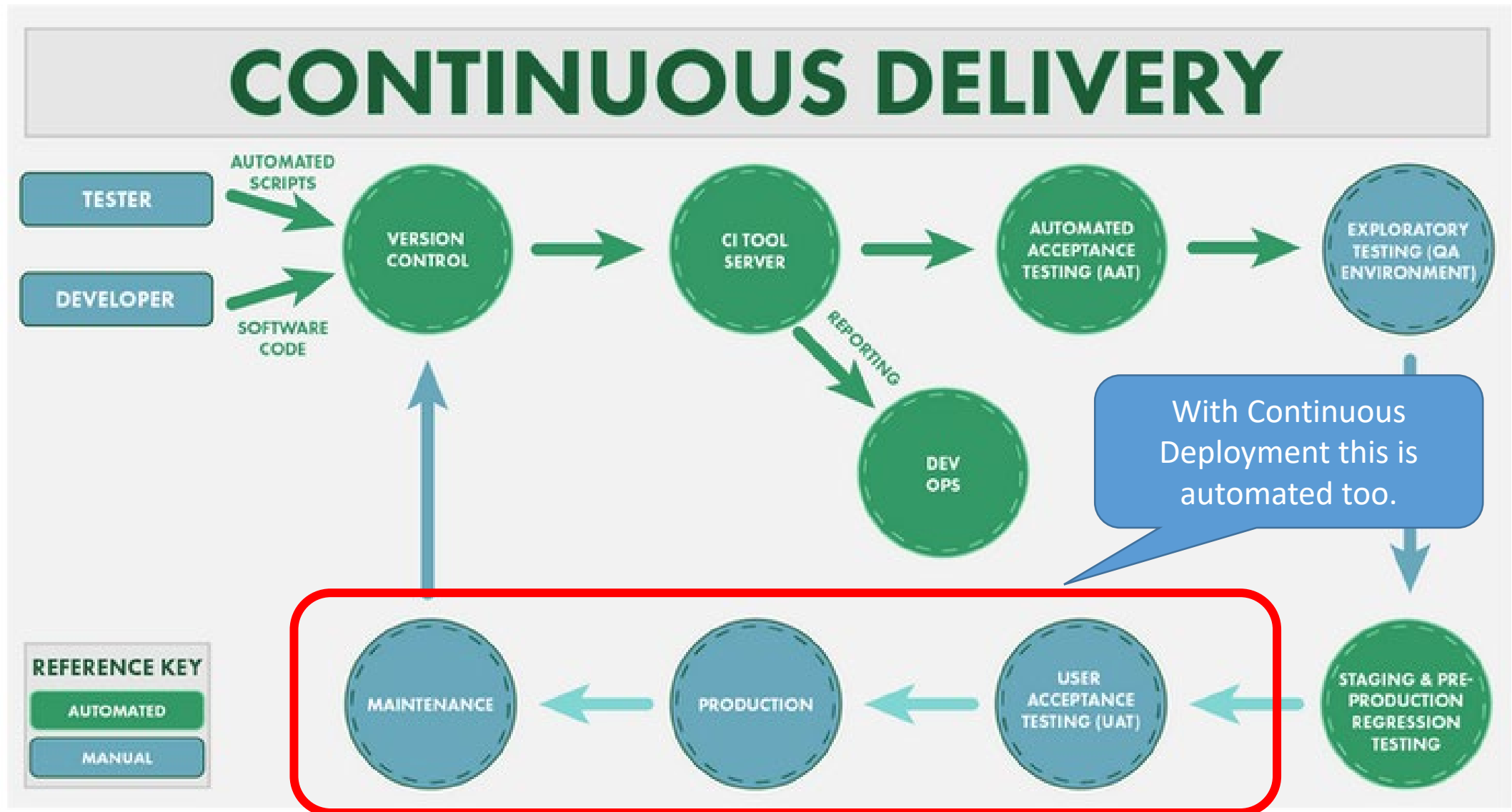
- Large companies – to keep the process lean, there may be separate DevOps teams for different products
- Medium-size companies – may adopt aspects of DevOps (automated testing) but not the whole thing
- Small companies – don't have enough people so developers are responsible for testing and deployment. Monitoring and Operations may not be supported

DevOps tools (incomplete list)

- Testing:
 - Selenium
 - Appium
- API dev:
 - Postman
- Code repositories:
 - Github, BitBucket, Gitlab
- Deployment
 - Jenkins, TeamCity, TravisCI
- Configuration
 - Puppet, Chef, Ansible Tower
- Orchestration
(coordinate automated DevOps and CI systems)
 - Kubernetes
 - Cloudify
 - Capistrano
- Monitoring
 - AmonOne
 - Scales
- Containerisation (later...)
 - Docker
 - Resin.io
 - CoreOS

Continuous Delivery and Deployment 2009

- Automated or manual deployment is added to a CI pipeline. Deployment can happen in parallel with developers adding features, testing and fixing bugs in the next deployment.
- **Continuous Delivery** is where the current build could be deployed but the actual deployment step is controlled **manually**.
- **Continuous Deployment** is where the current build is **automatically** deployed.



Microsoft:

<https://azure.microsoft.com/en-au/overview/continuous-delivery-vs-continuous-deployment/>

- Delivery "automatically placed in the desired environment and deployed."
- Deploy: "The trigger between the develop and deliver phases is automatic"
- Both sound like the same thing? Doesn't help.

Microsoft:

<https://azure.microsoft.com/en-au/overview/continuous-delivery-vs-continuous-deployment/>

Continuous Delivery or Continuous Deployment?

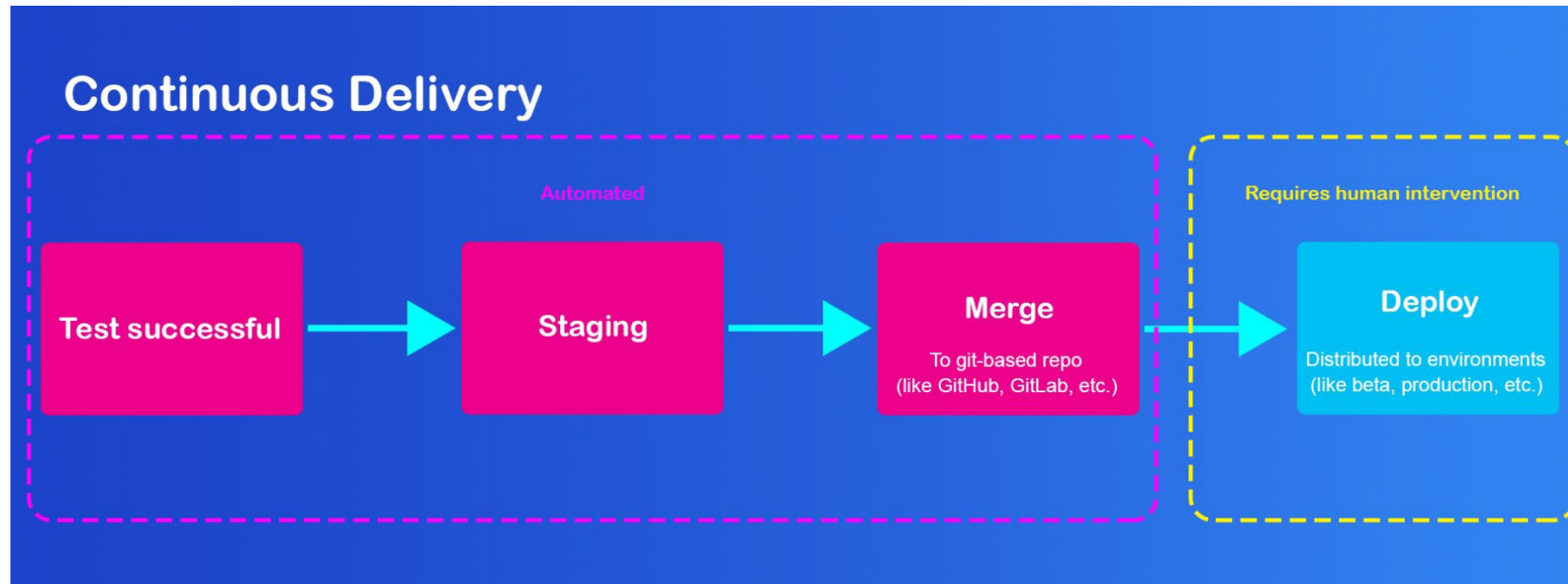
MS say Ask yourself:

- Can you deploy without approval from stakeholders? **Deploy**
- Do your system and gating requirements allow for end-to-end automation? **Deploy**
- Can you expose your customers to production changes a little at a time? **Deploy**
- Does your organisation respond to errors in production quickly? **Deploy**
- **If any of these answers are NO, do Continuous Delivery, and get approval**

CodeMagic.io:

<https://blog.codemagic.io/difference-between-continuous-deployment-and-continuous-delivery/>

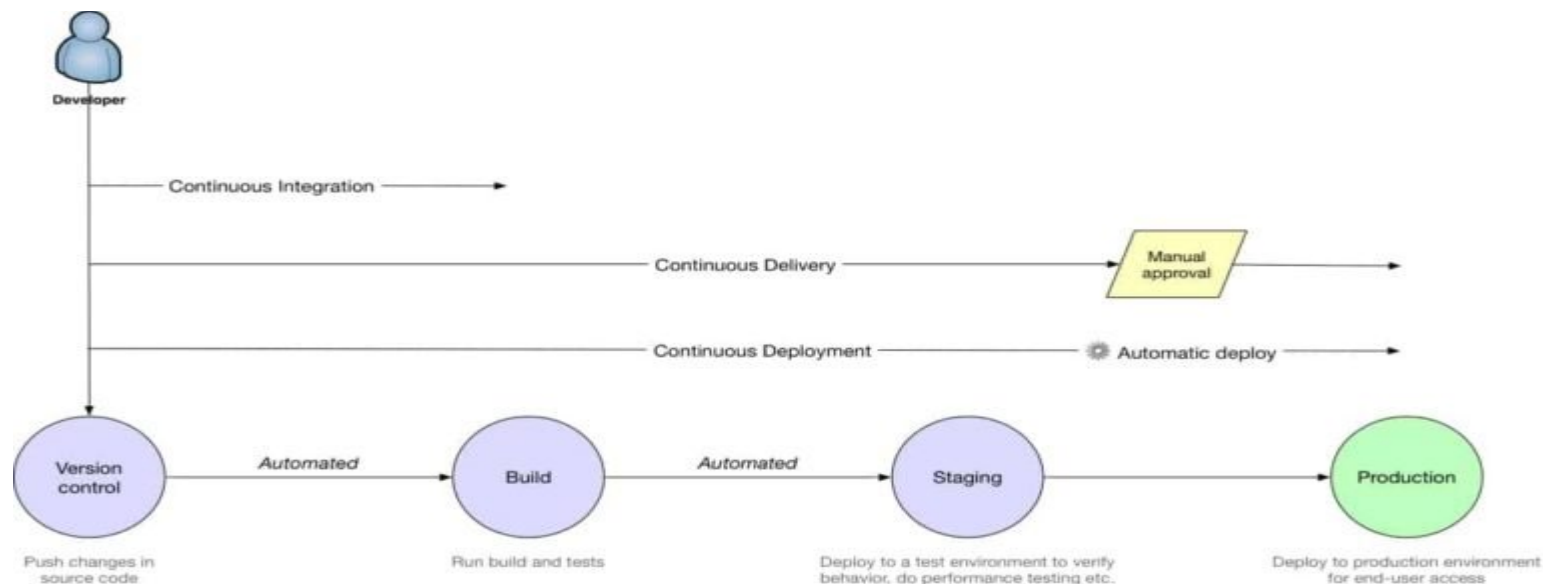
- Delivery: "deploy your changes at any time with just **one click**."
- Deploy: "if everything checks out in testing, the changes are deployed automatically"



SemaphoreCI:

<https://semaphoreci.com/blog/2017/07/27/what-is-the-difference-between-continuous-integration-continuous-deployment-and-continuous-delivery.html>

- "**Continuous Delivery (CD)** is a practice of automating the entire software release process.
- "**Continuous Deployment** is a step up from Continuous Delivery in which every change in the source code is deployed to production automatically, without explicit approval from a developer."



CI/CD 2015

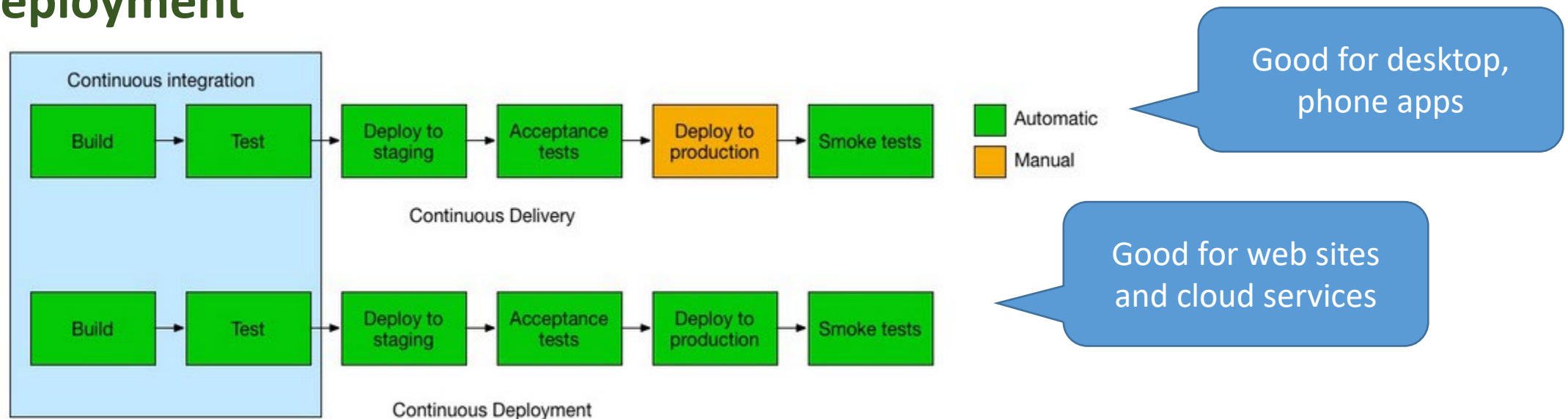
- Combine Continuous Integration tools with Continuous Delivery
- Builds on CI, CD and DevOps philosophies
 - Rapid feedback
 - Code branches are viewed as experiments – can be easily abandoned
 - Smaller changes mean bugs are easier to fix
 - Continuous build pipeline attached to continuous Deployment or Delivery pipeline
- Benefits
 - Very rapid, low-risk software updates
 - Identical staging/prod. environments (Blue/Green) which can be switched from test to live (zero downtime)
- But requires investment in automation tools, duplicate environments

CI/CD tools

- TeamCity
- Jenkins
- Travis CI
- GoCD
- Bamboo
- GitLab CI
- Circle CI
- Codeship
- ...

I'm still confused...

- CI = automated build and test
- CI/CD delivery = Automated everything except deployment
- Continuous Deployment = everything automated **including deployment**



CI/CD, DevOps and Mobile

- The processes are much the same for mobile devices except:
- Technology fragmentation
 - Extreme proliferation of platforms (hardware and service providers) compared to desktops, servers and cloud systems
- Back-end systems
 - Phone apps usually rely on server/cloud systems which may not be under the control of app developers – heavy reliance on SOA architectures and contracts
- Update processes
 - Users must choose to download/install updates. ISPs/Telcos must cooperate
- Instrumentation
 - Poor compared to web/cloud platforms

CI/CD, DevOps and Mobile

- Therefore:
 - CI/CD very difficult to implement
 - CD (Continuous Deployment) impossible
 - DevOps quite practical because it does not require deployment
- Solution:
 - Don't deploy phone apps
 - Deploy web apps and let users use their web browsers to access web site
 - Use HTML5 and scripting tricks to make web app behave like a phone app.
 - Allows CI/CD, CD to work for phone users.

CI/CD, DevOps and Games

- Indie development similar to DevOps:
 - Small teams
 - Dev, test, deploy, manage all in one team
- CI difficult because testing difficult to automate (hence Easter eggs/cheat codes to speed up manual testing)
- Continuous Deployment
 - works well for subscription services (Steam) because code is only deployed to game servers
 - portable device games – CD impossible (like mobiles)
 - Consoles - on-line updates possible.

DevSecOps 2012

- DevOps + security testing
- Tradition practice is to do security testing just before or after deployment. Many small companies don't even do this, waiting for cybersecurity attacks (DOS, data theft, defacement, account hijacks) before investing in security testing.
- Security testers are often viewed as the equivalent of Quality Assurance/Control in Manufacturing – the natural enemy of Production.
- DevSecOps aims to correct this culture by educating developers in safe coding practices and embedding automated security testing early in code production

DevSecOps attributes

- Embed security testing and practices into all parts of the pipeline
 - Coding (secure design patterns, best practice coding, “secure by design”)
 - Repositories (protected from malicious hackers)
 - Encrypted communications between repo, CI, CD, Prod. servers
- Automated security testing during development and integration
- Operations Monitoring to detect attacks
- Shared responsibility for security
- Cost savings and productivity gains
 - Fewer patches, less downtime, less reputational damage

DevSecOps

- DevOps team need to understand why security is their problem too.
- Dev teams need serious training. It is not enough to get the software working to spec. It must be resilient to attacks and mis-use.
- Automated security testing much slower than CI testing, and will not provide complete assurance.



People

People are the starting point of the DevSecOps implementation. Through ensuring proper training and restructuring of teams security will become a frame of mind rather than a hindrance.



Processes

DevSecOps aims to align and implement processes common to an enterprise to facilitate cooperation and achieve more secure development processes as a whole.

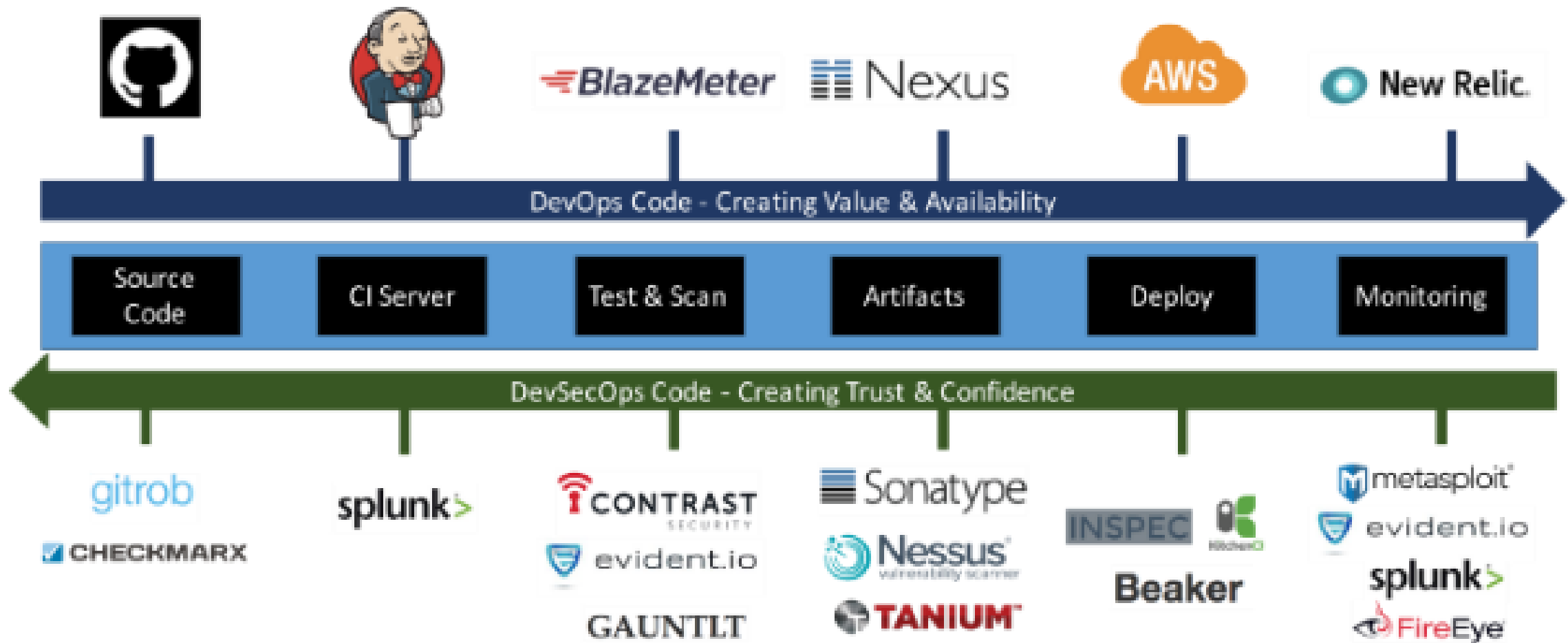


Technology

Technologies enable people to execute DevSecOps processes, which aim to reduce the enterprise attack surface and enable effective management of the technical security debt.

<https://www.devseccon.com/wp-content/uploads/2017/07/DevSecOps-whitepaper.pdf>

DevSecOps Tools



Source unknown

DevSecOps tools

- Checkmarx (integrates into DevOps)
- Gitrob (checks GitHub for blacklisted vulnerable files)
- Splunk Enterprise Server (CI)
- Evident.io (Ops monitoring)
- Sonatype (protects dev pipeline)
- Gauntlt (deployment and testing)
- Inspec (deployment and testing)
- Nessus (vulnerability scanner)
- None of these tools provide a full DevSecOps pipeline.

DevSecOps

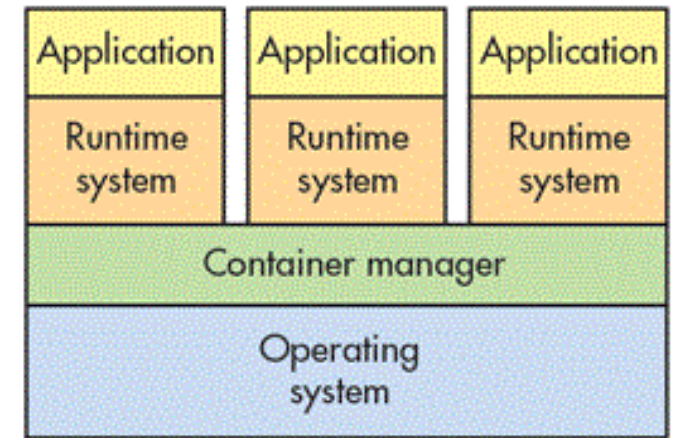
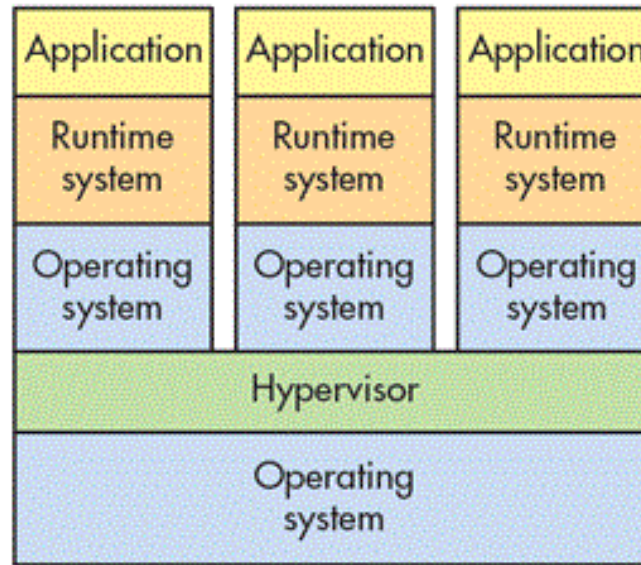
- Compared to DevOps, DevSecOps is immature and not well-adopted.

Containers (1979, 2013)

- Ancient Unix technology (e.g. chroot 1979) used to isolate an application/user from the operating system.
- Modern examples are:
 - Linux jails
 - Java and .NET virtual machines
 - VMWare, XenServer, KVM, QEMU, Hypervisors, JeOS,
 - Cloud platforms
 - Linux containers (LXC), Docker

Containers run under a Container Manager...

- Unlike VMs, the container uses the container manager to perform host OS functions
- Apps deployed as containers are platform-independent, smaller than VMs



<http://www.reply.com/solidsoft-reply/en/content/comparing-container-services-on-microsoft-azure>

Containers - advantages

- Portable, platform-independent applications
- Lighter weight compared to VMs
- Guest Isolation
- Resource sharing
- Rapid spin-up, shut down
- Scalable (like AWS, Azure, cloud)
- Simple deployment by “Orchestration” (e.g. Docker, Kubernetes)
- Can be deployed to IoT devices
- Suitable for micro-services

HowTo: Docker*

1. Go to <https://hub.docker.com> and create an account.
2. Respond to the verification e-mail.
3. Log in, download and install the Docker client on your laptop. Get the community edition from <https://www.docker.com/community-edition>
4. Launch Docker on your laptop – look for the “whale” icon to see if it is running.
5. Open a text console window (CUI), verify Docker version:

```
docker --version
```

*You will need admin rights to your laptop

HowTo: Docker

6. Try running hello-world – Docker will fail, download it and then succeed:

```
docker run hello-world
```

7. List the containers available:

```
docker container ls -all
```

Further steps are in this walkthrough on Canvas:
[docker walkthrough.pdf](#)

Next week – DevOps in detail